TRANSACCIONES Y PROPIEDADES ACID

Diseño de Base de Datos · Cap. 14

Ingeniería en Informática · 2.º año · Universidad Católica de Córdoba · Silberschatz, Korth & Sudarshan

Agenda

0. **Apertura — ¿por qué hace falta el concepto de transacción?**
1. **Propiedades ACID — atomicidad, consistencia, aislamiento, durabilidad**
2. **Estados de una transacción — del begin al commit/abort**
3. **Concurrencia, anomalías, planificaciones y secuencialidad**
4. **Recuperabilidad y planificaciones sin cascada**
5. **Niveles de aislamiento SQL — cuatro grados de protección**
6. **Cierre — síntesis y lecturas del capítulo**

Capítulo "teórico" del trío 14-15-16. Definimos las garantías que va a implementar el motor en los próximos dos capítulos.

El problema

Una **transferencia bancaria** de $50 entre cuentas A y B son dos operaciones:

```sql
-- Paso 1: debitar de A
UPDATE cuentas SET saldo = saldo - 50 WHERE id = 'A';

-- Paso 2: acreditar en B
UPDATE cuentas SET saldo = saldo + 50 WHERE id = 'B';
```

⚠ **¿QUÉ PASA SI EL SISTEMA CAE ENTRE PASO 1 Y PASO 2?**

Se perdieron $50. La base queda en un estado **inconsistente** — los datos por sí solos no son suficientes, necesitamos una **unidad de trabajo**.

✦ **En SIGUE:** una **inscripción a comisión** tiene el mismo problema. Hay que descontar el cupo (`UPDATE comisiones SET cupo = cupo - 1`) Y crear la fila en `inscripciones`. Si el sistema cae en el medio, queda un cupo descontado sin alumno — o un alumno sin lugar real.

Una transacción agrupa varias operaciones en una unidad lógica que el motor ejecuta *como si fuera una sola*: o se hacen todas o ninguna.

01

Propiedades ACID

Cuatro garantías que ofrece el motor

Las cuatro letras

Propiedad	Significado	La garantiza
Atomicidad	Todas las operaciones de la transacción se reflejan en la BD, o ninguna. Si algo falla, los cambios parciales se deshacen.	Recuperación (cap. 16)
Consistencia	Si la BD parte de un estado consistente y la transacción es lógicamente correcta, la BD termina en otro estado consistente.	Programador + motor (CHECK, FK…)
Aislamiento	Cada transacción ve la BD como si fuera la única ejecutándose. La concurrencia no debe filtrarse.	Control de concurrencia (cap. 15)
Durabilidad	Una vez que el motor confirma el commit, los cambios sobreviven a cualquier fallo posterior del sistema.	Recuperación + almacenamiento estable

IDEA FUERZA

ACID no es un detalle de implementación: es un **contrato** entre el motor y la aplicación. El programador puede asumir que estas cuatro garantías se cumplen, sin código adicional.

ACID en la transferencia

Volvamos a la transferencia de $50 entre A y B. ¿Qué garantiza cada propiedad?

ATOMICIDAD

Si el motor cae entre los dos UPDATE, al reiniciar deshace el primero. No quedan $50 perdidos en el limbo.

CONSISTENCIA

La suma total de saldos (regla del negocio) se conserva: la BD termina con la misma plata, redistribuida.

AISLAMIENTO

Si otra consulta corre en paralelo, no ve el estado intermedio: o ve los $50 todavía en A, o ya en B. Nunca "en ningún lado".

DURABILIDAD

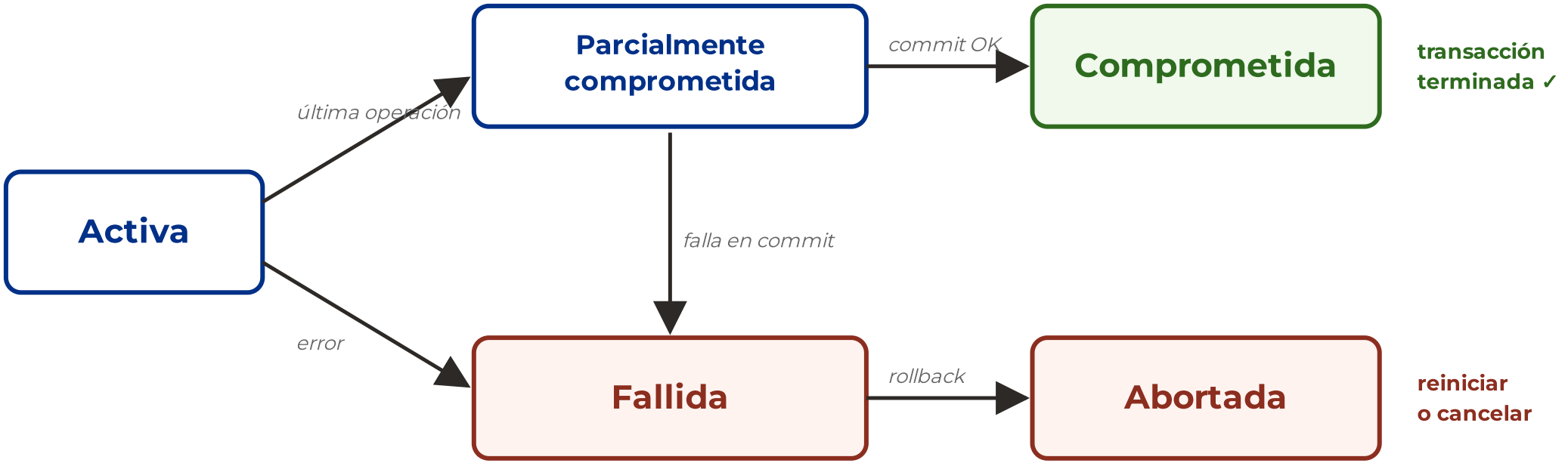
Una vez que el motor responde "commit OK", aunque el servidor se incendie en el segundo siguiente, la transferencia está hecha.

✓ **RECORDAR**

Sin transacciones, el programador tendría que escribir él mismo todo el código de "deshacer si falla algo" — y hacerlo correctamente bajo concurrencia es **casi imposible**.

Estados de una transacción

Toda transacción atraviesa una **máquina de estados**. El motor sabe en cuál está cada una en cada instante.



⚠ **Una vez Comprometida** no se puede revertir con `ROLLBACK`. Si después hay que "deshacer" algo, hay que ejecutar una **nueva** transacción que compense (p. ej. devolver el cupo, anular la nota cargada).

Commit y rollback en SQL

Una transacción empieza con la primera sentencia tras un commit/rollback y termina explícitamente:

```sql
-- transferencia con manejo explícito
BEGIN;
  UPDATE cuentas SET saldo = saldo - 50 WHERE id = 'A';
  UPDATE cuentas SET saldo = saldo + 50 WHERE id = 'B';
COMMIT;                -- confirma los cambios

-- si algo sale mal antes del COMMIT
ROLLBACK;              -- descarta todos los cambios de la TX
```

⚠ AUTOCOMMIT

Por defecto, muchos clientes SQL **auto-comprometen cada sentencia**: cada `UPDATE` es su propia transacción. Para agrupar varias, hay que abrir explícitamente con `BEGIN` (o desactivar autocommit en la conexión).

En JDBC: `connection.setAutoCommit(false)`. En psql: `BEGIN`. En MySQL: `START TRANSACTION`.

02

Concurrencia y sus problemas

¿Qué se rompe cuando varias transacciones corren en paralelo?

¿Por qué no ejecutarlas en serie?

Sería más simple: empezar T1, terminarla, empezar T2, terminarla. Pero el motor permite ejecutarlas **intercaladas**. ¿Por qué?

THROUGHPUT

Mientras T1 espera disco (E/S lenta), la CPU está ociosa. T2 podría aprovecharla. Sin concurrencia, el sistema desperdicia recursos.

LATENCIA

Una transacción larga (un reporte que demora 30 segundos) bloquearía a las cortas (una consulta de saldo de 50 ms). Las cortas terminarían esperando 30 s. Inaceptable.

⚠ PERO OJO

La concurrencia introduce **anomalías**: comportamientos que la ejecución en serie nunca tendría. El control de concurrencia (cap. 15) es el subsistema que las evita — al costo de algo de paralelismo.

Trade-off central: más paralelismo ↔ menos garantías ↔ posibles anomalías.

Las cuatro anomalías clásicas

LECTURA SUCIA (`DIRTY READ`)

T1 lee un dato que T2 modificó pero **no comprometió todavía**. Si T2 aborta, T1 leyó un valor "fantasma" que nunca existió oficialmente.

LECTURA NO REPETIBLE (`NON-REPEATABLE READ`)

T1 lee Q. Mientras tanto T2 modifica y compromete Q. Cuando T1 lee Q de nuevo, ve otro valor. La misma consulta da resultados distintos.

LECTURA FANTASMA (`PHANTOM READ`)

T1 lee un *conjunto de filas* que cumple un predicado. T2 inserta filas nuevas que cumplen el predicado. T1 vuelve a leer y aparecen filas "fantasma" que no estaban antes.

ACTUALIZACIÓN PERDIDA (`LOST UPDATE`)

T1 y T2 leen Q. Ambas calculan un nuevo valor y lo escriben. La segunda escritura sobrescribe a la primera — un cambio se perdió silenciosamente.

LO QUE HAY QUE RETENER

Estas anomalías **no son bugs del motor**: son consecuencias inevitables de la concurrencia. Lo que el motor ofrece son **niveles de aislamiento** con distintos compromisos entre paralelismo y protección.

Ejemplo: dirty read (lectura sucia)

T_1 intenta inscribir al Alumno X a Comisión-001 (descuenta el cupo) pero **falla** y aborta. **T_2** es el dashboard de ocupación que muestra el cupo a Bedelía. Estado inicial: `cupo = 30`.

T_1 (intento de inscripción)	T_2 (dashboard de ocupación)
`leer(cupo)` → 30	
`cupo := cupo - 1`	
`escribir(cupo = 29)`	
	`leer(cupo)` → 29 ⚠ sucio
	mostrar "29 lugares libres"
💥 T_1 falla — `ROLLBACK`	
(cupo vuelve a 30)	

⚠ ANOMALÍA

T_2 leyó un valor **no comprometido** (29). Cuando T_1 aborta, ese 29 nunca existió oficialmente — pero T_2 ya lo mostró. **El dashboard miente:** dice 29 libres cuando realmente hay 30.

✓ CÓMO SE EVITA

Nivel de aislamiento `read committed` o superior. Una TX **solo lee datos comprometidos**: T_2 habría esperado hasta que T_1 hiciera commit o rollback antes de poder leer.

✦ **Por qué importa en SIGUE:** los dashboards, mails de notificación y reportes nunca deben mostrar datos en tránsito. Sin `read committed`, alumnos y docentes verían información que se desvanece.

Ejemplo: non-repeatable read (lectura no repetible)

T_1 calcula la beca del Alumno X — necesita el saldo en dos momentos. **T_2** registra un pago de matrícula del mismo alumno. Estado inicial: `saldo = 1000`.

T_1 (cálculo de beca)	T_2 (registro de pago)
`leer(saldo)` → 1000	
`beca := 1000 * 0.1 = 100`	
	`leer(saldo)` → 1000
	`saldo := saldo + 500`
	`escribir(saldo = 1500)`
	`COMMIT` ✓
`leer(saldo)` → 1500 ⚠ cambió!	
¿beca sobre 1000 o 1500?	

⚠ ANOMALÍA

T_1 leyó el saldo dos veces durante su ejecución y obtuvo **valores distintos** (1000 → 1500). T_2 se metió en el medio, modificó y comprometió. La consistencia interna de T_1 se rompe — su cálculo depende de cuál lectura use.

✓ CÓMO SE EVITA

Nivel `repeatable read` o superior. Una vez que T_1 leyó `saldo`, el motor garantiza que **cualquier lectura posterior en T_1 vea el mismo valor**. T_2 habría esperado a que T_1 termine.

✦ **Por qué importa en SIGUE:** cualquier proceso que tome decisiones financieras (becas, exenciones, descuentos) usando varias lecturas del mismo dato necesita `repeatable read`.

Ejemplo: phantom read (lectura fantasma)

T_1 es una auditoría que cuenta los inscriptos a Comisión-001 dos veces durante su ejecución. **T_2** es una nueva inscripción que *cumple el predicado*. Estado inicial: 25 inscriptos.

T_1 (auditoría)	T_2 (nueva inscripción)
`SELECT COUNT(*) FROM`	
`  inscripciones`	
`  WHERE comision = 001`	
→ 25	
	`INSERT INTO inscripciones`
	`  (alumno_Y, comision = 001)`
	`COMMIT` ✓
`SELECT COUNT(*) FROM ...` → **26** ⚠ ¡26!	
Apareció una fila "fantasma"	

⚠ ANOMALÍA

T_1 ejecutó **el mismo predicado** dos veces y obtuvo cantidades distintas (25 → 26). El dato fantasma es una **fila nueva** que cumple el WHERE pero no existía antes. *Non-repeatable read* bloquea cambios sobre filas existentes; phantom permite inserciones nuevas que entran al resultado.

✓ CÓMO SE EVITA

Nivel `serializable` — el más estricto. El motor garantiza que **el conjunto de filas que cumplen el predicado no cambia** durante T_1. Se logra con bloqueos de rango (cap. 15) o serializable snapshot isolation.

✦ **Por qué importa en SIGUE:** reportes de cupos, listados de inscriptos para actas, conteos de morosos — cualquier consulta agregada que se ejecute más de una vez en la misma TX necesita `serializable`.

Ejemplo: lost update (actualización perdida)

T_1 y **T_2** son dos puntos de inscripción (p. ej. Bedelía y el portal web) anotando alumnos a Comisión-001 al mismo tiempo. Estado inicial: `cupo = 30`.

T_1 (inscribe Alumno X)	T_2 (inscribe Alumno Y)
`leer(cupo)` → 30	
	`leer(cupo)` → 30
`cupo := cupo - 1`	
`escribir(cupo = 29)`	
`COMMIT` ✓	
	`cupo := cupo - 1`
	`escribir(cupo = 29)` ⚠ pisa a T_1
	`COMMIT` ✓
Final: cupo=29, dos inscriptos, solo un cupo descontado	

⚠ ANOMALÍA

Ambas TX leyeron `cupo=30`, ambas calcularon 29, ambas escribieron 29. **La actualización de T_1 se perdió** — la escritura de T_2 la sobrescribió. El cupo bajó de 30 a 29 cuando deberían ser dos descuentos (cupo=28).

✓ CÓMO SE EVITA

Nivel `repeatable read` o superior. También con `SELECT ... FOR UPDATE`, que bloquea la fila apenas se lee — T_2 esperaría a que T_1 haga commit antes de poder leer.

✦ **Por qué importa en SIGUE:** es la anomalía más peligrosa para los contadores (cupos, stocks, saldos). La comisión queda **sobrevendida** — un alumno se entera el primer día de clase que no tiene asiento.

Planificaciones (schedules)

Cuando varias transacciones corren a la vez, el motor produce una **planificación**: la secuencia ordenada de operaciones de todas las TXs activas. Hay dos formas de armarla:

Serial — segura, lenta



Intercalada — rápida, puede romper



Tipos de planificación que importan en la materia:

Tipo	Definición
Serial	T1 entera, después T2 entera. Aislamiento perfecto, sin paralelismo.
Secuenciable	Intercalada, pero el resultado equivale a *alguna* serial. **Lo que el motor garantiza.**
Recuperable	Si T_j lee un dato de T_i, entonces T_i debe hacer commit *antes* que T_j. Evita que T_j termine confiando en datos que después se desvanecen.
Sin cascada	Una TX solo lee datos ya comprometidos. Si una aborta, no arrastra a otras (no hay *cascading rollback*).

✦ **En SIGUE:** dos inscripciones simultáneas a Comisión-001 con cupo=30. La planificación *serial* termina con cupo=28 ✓. La *intercalada de arriba* (sin control) termina con cupo=29 — un alumno perdió su lugar.

¿Cómo verifica el motor que es secuenciable?

Dos operaciones están en **conflicto** si tocan el mismo dato Y al menos una es `write`:

	read	write
read	**sin conflicto**	**conflicto**
write	**conflicto**	**conflicto**

Solo read-read es seguro: dos TXs leyendo el mismo dato nunca se interfieren.

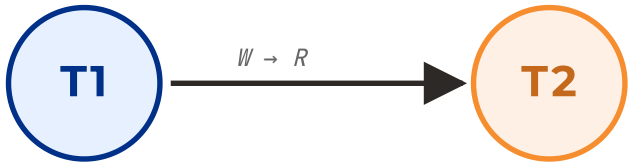
Grafo de precedencia:

- **Nodos** = transacciones de la planificación.
- **Arista** $T_i \rightarrow T_j$ si T_i tiene una operación previa a una conflictiva de T_j.

✓ **Teorema:** la planificación es secuenciable en conflicto ⟺ **el grafo es acíclico**.

✓ Acíclico — secuenciable

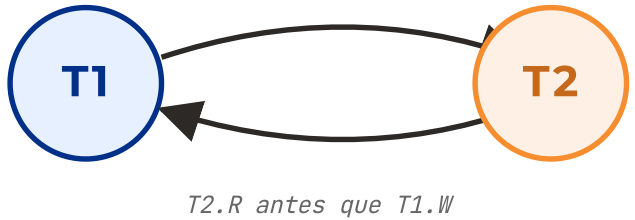
`Plan: T1.W(cupo) → T2.R(cupo) → T2.W(cupo)`



orden serial equivalente: T1 ; T2

✗ Con ciclo — NO secuenciable

`Plan: T1.R(cupo) → T2.R(cupo) → T1.W(cupo) → T2.W(cupo)`



✦ **En SIGUE:** el caso de la derecha es exactamente lo que pasó con las dos inscripciones de la slide anterior. El ciclo T1 ⇄ T2 demuestra que esa planificación **no equivale a ninguna ejecución serial** — por eso un alumno quedó sin lugar. 2PL (cap. 15) garantiza que el grafo nunca tenga ciclos.

Planificación serial — ejemplo del libro

T_1 transfiere 50 € de A a B. **T_2** transfiere el 10% de A a B. Estado inicial: **A = 1000 €**, **B = 2000 €**. Suma A+B = 3000 € (invariante a conservar).

Fig. 14.2 — Planificación 1: serial T_1 ; T_2

T_1	T_2
`leer(A)`	
`A := A - 50`	
`escribir(A)`	
`leer(B)`	
`B := B + 50`	
`escribir(B)`	
	`leer(A)`
	`temp := A * 0.1`
	`A := A - temp`
	`escribir(A)`
	`leer(B)`
	`B := B + temp`
	`escribir(B)`

✓ ESTADO FINAL

A = 855 €, **B = 2145 €**. Suma = **3000 €** ✓ — el invariante se conserva.

¿POR QUÉ ES CORRECTA?

T_1 se ejecuta entera antes de que T_2 empiece. Cada una corre *aislada*. No hay forma de que se interfieran. **Toda planificación serial es correcta por construcción.**

CUÁNTAS SERIALES HAY

Para *n* transacciones existen **n!** planificaciones seriales. La Fig. 14.3 muestra la otra opción: T_2 ; T_1 (también correcta, distinto estado final).

✦ **En SIGUE:** T_1 = inscripción a Comisión-001 (descontar cupo + sumar a total); T_2 = auditoría que reserva 10% como lugares de gracia. Ejecutadas en serie, el invariante (cupo + total) se mantiene.

Concurrente: ¿secuenciable o inconsistente?

Mismas T_1 y T_2 que la slide anterior. Lo que cambia es cómo se **intercalan** sus operaciones. No toda intercalación es válida.

✓ Fig. 14.4 — Planificación 3: secuenciable

T_1	T_2
leer(A)	
A := A - 50	
escribir(A)	
	leer(A)
	temp := A * 0.1
	A := A - temp
	escribir(A)
leer(B)	
B := B + 50	
escribir(B)	
	leer(B)
	B := B + temp
	escribir(B)

Final: **A=855, B=2145, suma=3000 ✓**. **Equivale a la serial $T_1;T_2$** — es **secuenciable**.

✗ Fig. 14.5 — Planificación 4: inconsistente

T_1	T_2
leer(A)	
A := A - 50	
	leer(A)
	temp := A * 0.1
	A := A - temp
	escribir(A)
	leer(B)
escribir(A)	
leer(B)	
B := B + 50	
escribir(B)	
	B := B + temp
	escribir(B)

Final: **A=950, B=2100, suma=3050 ✗**. T_2 leyó A y B antes que T_1 los escribiera; ambas sobrescriben — **se "ganaron" 50 €**. No equivale a ninguna serial.

✦ **En SIGUE:** análogo a dos inscripciones simultáneas a la misma comisión — la planificación de la izquierda termina con cupo correcto; la de la derecha pierde una actualización y el cupo no coincide con la cantidad real de inscriptos.

Conflicto entre operaciones — los 4 casos

Dos operaciones `I` (de T_i) y `J` (de T_j, con $i \neq j$) están en **conflicto** si operan sobre el mismo dato `Q` y al menos una es `escribir`:

#	I	J	¿Conflicto?	Por qué
1	`leer(Q)`	`leer(Q)`	**NO**	El orden no importa; ambas leen el mismo valor independientemente del orden.
2	`leer(Q)`	`escribir(Q)`	**SÍ**	Si I va antes, lee el valor viejo; si va después, lee el nuevo. El orden importa.
3	`escribir(Q)`	`leer(Q)`	**SÍ**	Igual que el caso 2, por simetría.
4	`escribir(Q)`	`escribir(Q)`	**SÍ**	El orden determina qué valor queda y qué lee la siguiente `leer(Q)`.

Las 3 condiciones del grafo de precedencia

Hay arista $\mathbf{T_i \rightarrow T_j}$ en el grafo si se cumple *al menos una* de estas condiciones (con la operación de T_i apareciendo **antes** que la de T_j):

#	Condición
1	T_i ejecuta `escribir(Q)` **antes** de que T_j ejecute `leer(Q)`
2	T_i ejecuta `leer(Q)` **antes** de que T_j ejecute `escribir(Q)`
3	T_i ejecuta `escribir(Q)` **antes** de que T_j ejecute `escribir(Q)`

✓ ORDEN TOPOLÓGICO

Si el grafo es acíclico, cualquier **orden topológico** es una serialización válida. Puede haber varios (Fig. 14.12 muestra dos para el mismo grafo).

DETECCIÓN DE CICLOS

Algoritmo de búsqueda en profundidad: **$O(n^2)$** con n = número de transacciones. Barato comparado con verificar todas las planificaciones.

Secuenciable en vistas — la versión generalizada

Definición más permisiva. Dos planificaciones *S* y *S'* son **equivalentes en vistas** si cumplen las **tres condiciones**:

#	Condición de equivalencia en vistas
1	Para cada dato Q, la transacción que hace la **primera lectura** de Q es la misma en S y S'.
2	Para cada `leer(Q)`, la transacción que **escribió** el valor leído es la misma en S y S'.
3	La transacción que hace la **última escritura** de cada Q es la misma en S y S'.

Conflicto vs vistas — cuadro comparativo

Criterio	Secuenciable en conflicto	Secuenciable en vistas
Definición	Equivalente por intercambio de operaciones no conflictivas	Equivalente en primera lectura, "lee de", y última escritura
Permisividad	Más estricta	Más permisiva — incluye *blind writes*
Verificación	**O(n²)** con grafo de precedencia	**NP-completo**
Uso en motores	**Sí** — lo que se implementa en la práctica	No — solo definición teórica

conflicto ⊂ vistas *— toda en conflicto es en vistas, pero no al revés.*

¿QUÉ ES UNA "ESCRITURA CIEGA"?

Una `escribir(Q)` en una TX que **no fue precedida** por una `leer(Q)` en esa misma TX. Una escritura "a ciegas". Aparecen en planificaciones que son secuenciables en vistas pero no en conflicto.

✓ POR QUÉ LOS MOTORES USAN CONFLICTO

Verificar conflicto es polinómico — basta construir el grafo y buscar ciclos. Verificar vistas es NP-completo: no escala a sistemas reales con miles de TXs concurrentes. La permisividad extra no compensa el costo.

Equivalencia en vistas — Condición 1: primera lectura

*"Para cada dato Q, la transacción que hace la **primera lectura** de Q es la misma en S y S'."*

✓ EQUIVALENTES — primera lectura: T_1 en ambas

S:

T_1	T_2
`leer(Q)` ← 1ra	
`escribir(Q)`	
	`leer(Q)`
	`escribir(Q)`

S':

T_1	T_2
`leer(Q)` ← 1ra	
	`leer(Q)`
`escribir(Q)`	
	`escribir(Q)`

✗ NO EQUIVALENTES — primera lectura distinta

S:

T_1	T_2
`leer(Q)` ← 1ra: T_1	
`escribir(Q)`	
	`leer(Q)`

S':

T_1	T_2
	`leer(Q)` ← 1ra: T_2
`escribir(Q)`	
`leer(Q)`	

✦ **En SIGUE:** si en S la consulta de Bedelía es la primera en leer el `cupo` y en S' lo lee primero el portal web, leen estados iniciales distintos y los cálculos posteriores divergen. **Las planificaciones no son intercambiables.**

Equivalencia en vistas — Condición 2: "lee de"

*"Para cada `leer(Q)`, la transacción que **escribió el valor leído** es la misma en S y S'."*

✓ EQUIVALENTES — T_2 lee de T_1 en ambas

S:

T_1	T_2
`escribir(Q=29)`	
	`leer(Q)`→`29` de T_1
	`escribir(Q)`

S':

T_1	T_2
`escribir(Q=29)`	
`commit`	
	`leer(Q)`→`29` de T_1
	`escribir(Q)`

✗ NO EQUIVALENTES — T_2 lee de fuentes distintas

S:

T_1	T_2
`escribir(Q=29)`	
	`leer(Q)`→`29` de T_1

S':

T_1	T_2
	`leer(Q)`→`30` valor inicial
`escribir(Q=29)`	

✦ **En SIGUE:** T_2 es el cálculo de la beca y T_1 registra el pago. En S, T_2 lee el saldo *después* del pago; en S', lo lee *antes*. **T_2 termina calculando becas distintas** según qué planificación se haya ejecutado.

Equivalencia en vistas — Condición 3: última escritura

*"La transacción que hace la **última escritura** de cada Q es la misma en S y S'."*

✓ EQUIVALENTES — última escritura: T_2 en ambas

S:

T_1	T_2
`escribir(Q)`	
	`escribir(Q)` ← última

S':

T_1	T_2
`escribir(Q)`	
	`leer(Q)`
	`escribir(Q)` ← última

✗ NO EQUIVALENTES — última escritura distinta

S:

T_1	T_2
`escribir(Q)`	
	`escribir(Q)` ← última: T_2

S':

T_1	T_2
	`escribir(Q)`
`escribir(Q)` ← última: T_1	

✦ **En SIGUE:** el valor que queda en la BD es el de la última escritura. Si en S el cupo final lo escribe el portal web y en S' lo escribe Bedelía con un valor distinto, **la BD queda con estados diferentes** y futuras consultas leen valores diferentes.

03

Recuperabilidad y atomicidad

¿Qué pasa con T_2 si T_1 aborta después de que T_2 leyó algo de ella?

Planificación recuperable

Combinar concurrencia con atomicidad agrega un problema nuevo: si T_j leyó un dato escrito por T_i, y luego **T_i aborta**, ¿qué pasa con T_j?

✓ DEFINICIÓN

Una planificación es **recuperable** si, para todo par T_i, T_j donde T_j lee un dato escrito por T_i, el **`commit` de T_i aparece antes del `commit` de T_j**.

✗ Fig. 14.14 — Planificación 9 (NO recuperable)

T_6	T_7
`leer(A)`	
`escribir(A)`	
	`leer(A)` ← lee A de T_6
	`commit` ← comprometió ANTES que T_6
`leer(B)`	
💥 T_6 falla aquí	

⚠ POR QUÉ NO SE PUEDE RECUPERAR

T_7 se comprometió leyendo un valor escrito por T_6. Pero T_6 falla — su escritura nunca debió existir. T_7 confió en un valor que se desvanece, y **ya no se puede revertir T_7** (ya comprometió). La atomicidad de T_6 se rompe.

✓ LA SOLUCIÓN

Demorar el commit de T_7 hasta que T_6 comprometa. Garantizar recuperabilidad puede **retrasar commits**, pero no hay alternativa.

✦ **En SIGUE:** T_6 inscribe a un alumno y descuenta el cupo de Comisión-001. T_7 envía el mail de confirmación leyendo ese cupo y comprometiéndose. Si T_6 falla, el alumno ya recibió el mail por un lugar que no tiene.

Retroceso en cascada (cascading rollback)

Aun siendo recuperable, abortar una TX puede obligar a abortar **todas las que leyeron de ella**, las que leyeron de esas, etc. — un efecto dominó muy costoso.

✗ Fig. 14.15 — Planificación 10

T_8	T_9	T_{10}
`leer(A)`		
`leer(B)`		
`escribir(A)`		
	`leer(A)` ← de T_8	
	`escribir(A)`	
		`leer(A)` ← de T_9
💥 T_8 falla	↳ T_9 aborta	↳ T_{10} aborta

✓ DEFINICIÓN DE "SIN CASCADA"

Una planificación es **sin cascada** (*cascadeless*) si para todo par T_i, T_j donde T_j lee un dato escrito por T_i, el **`commit` de T_i aparece antes de la `leer` de T_j**.

*Es decir: una TX **solo lee datos ya comprometidos**.*

SIN CASCADA ⟹ RECUPERABLE

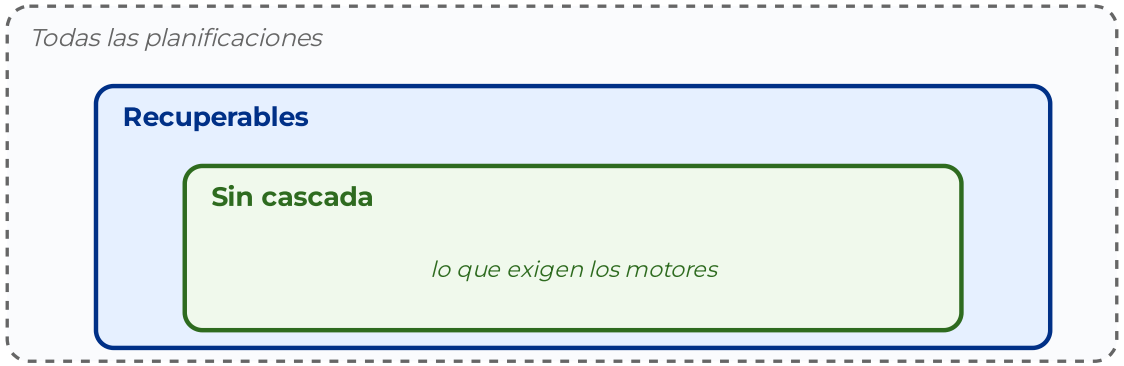
Toda planificación sin cascada es recuperable (al revés no vale). Los motores normalmente exigen **sin cascada** — requisito más fuerte que "solo secuenciable", con algo de pérdida de paralelismo a cambio.

✦ **En SIGUE:** T_8 inscribe (escribe cupo). T_9 es una segunda inscripción que lee ese cupo no comprometido. T_{10} es un reporte de ocupación que lee el cupo de T_9. Si T_8 aborta, se caen T_9 y T_{10} — todo el trabajo posterior se pierde.

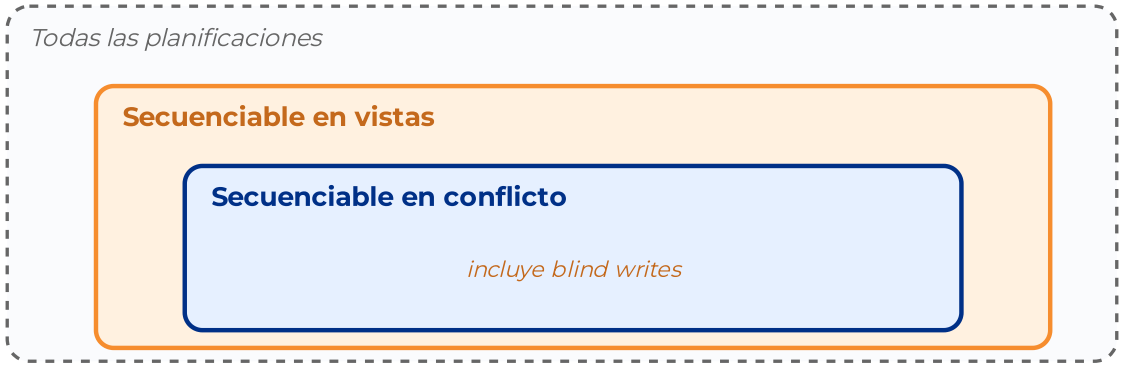
Cuadro síntesis — los 4 tipos de planificación

Tipo	Qué garantiza	Cómo se verifica	Ejemplo SIGUE
Serial	Aislamiento total, cero anomalías	Por construcción (una TX entera tras otra)	Inscribir un alumno completo antes del siguiente
Secuenciable	Mismo resultado que *alguna* serial	Grafo de precedencia acíclico (**$O(n^2)$**)	Cupo final igual al de procesar inscripciones en serie
Recuperable	No confiar en datos que luego desaparecen	`commit` de T_i antes del `commit` de T_j	No mandar mail hasta que la inscripción esté comprometida
Sin cascada	Que un aborto no arrastre a otras TXs	`commit` de T_i antes de la `leer` de T_j	Cada proceso lee solo cupos ya comprometidos

Jerarquía de inclusión



sin cascada ⊂ recuperable ⊂ todas



conflicto ⊂ vistas

Los motores reales buscan planificaciones **secuenciables en conflicto Y sin cascada** — la intersección. Es lo que verán en el cap. 15 con **strict 2PL**.

04

Niveles de aislamiento

Cuatro grados de protección, definidos por SQL

Cuatro niveles, cuatro trade-offs

SQL estandariza cuatro niveles. Cada uno **permite** ciertas anomalías:

Nivel	Dirty read	Non-repeatable	Phantom	Concurrencia
`read uncommitted`	**permite**	**permite**	**permite**	máxima
`read committed`	**evita**	**permite**	**permite**	alta
`repeatable read`	**evita**	**evita**	**permite**	media
`serializable`	**evita**	**evita**	**evita**	baja

POR DEFECTO

La mayoría usa `read committed` (Oracle, PostgreSQL, SQL Server). MySQL/InnoDB usa `repeatable read`. SQLite usa `serializable`.

✓ IDEA

Más estricto = más protección = menos paralelismo. El programador **elige** el nivel según el riesgo que puede tolerar.

Cómo se configura el nivel

Antes de empezar la transacción, se indica el nivel deseado:

```sql
-- para la próxima transacción
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN;
  SELECT saldo FROM cuentas WHERE id = 'A';
  -- ... lógica ...
```

Variantes equivalentes en distintos motores:

```sql
-- PostgreSQL / SQL Server
BEGIN ISOLATION LEVEL REPEATABLE READ;

-- MySQL
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

✓ REGLA PRÁCTICA

Usar el **nivel más estricto** que el sistema soporte sin perder demasiado rendimiento. Subir de nivel sólo cuando se detecte una anomalía concreta — bajarlo "por las dudas" es como apagar el airbag para ahorrar peso.

Cierre de la clase

- Una **transacción** es una unidad lógica de trabajo con propiedades **ACID**.
- La **atomicidad** garantiza "todo o nada"; la **durabilidad**, que el commit sobrevive a fallos.
- Una transacción atraviesa estados: *activa* → *parcialmente comprometida* → *comprometida* (o *fallida* → *abortada*).
- SQL: `BEGIN`, `COMMIT`, `ROLLBACK`. Cuidado con el **autocommit**.
- La concurrencia es necesaria por **throughput** y **latencia**, pero introduce anomalías.
- Cuatro anomalías: *dirty read*, *non-repeatable read*, *phantom read*, *lost update*.
- **Planificaciones**: serial, secuenciable, recuperable, sin cascada. El grafo de precedencia verifica secuencialidad.
- SQL define cuatro **niveles de aislamiento** con distintos trade-offs entre protección y paralelismo.
- Cap. 15: *cómo* el motor implementa estos niveles. Cap. 16: *cómo* garantiza atomicidad ante fallos.

📖 **LECTURAS — SILBERSCHATZ, CAP. 14**

- **14.1** Concepto y propiedades ACID · **14.4** Atomicidad y durabilidad (intro WAL)
- **14.5** Aislamiento — anomalías de concurrencia
- **14.6** Secuencialidad — planificaciones serial, secuenciable, grafo de precedencia
- **14.7** Aislamiento y atomicidad — planificaciones recuperables y sin cascada
- **14.8** Niveles de aislamiento SQL · **14.10** Transacciones como sentencias de SQL